Campus Lost & Found Intelligent Matching System

Final relational schema

DBMS Experiential Learning Level 3 | 2 October 2026

Member	SRN
Harshit Chandak	PES2UG24CS185
Hamza Shabir Sahapurwala	PES2UG24CS177

Canonical fresh-install schema from actual database metadata. Ten domain and two infrastructure tables are documented separately, including every column, key, CHECK, default, revision and generated uniqueness field. PK means primary key; FK means foreign key. A dash in Default means no non-NULL default is reported. Source SQL uses uppercase table names; Windows metadata uses equivalent lowercase names. Fresh identities are unsigned; compatibility retains original signed types. Dates persist as UTC and display in IST. The approved conceptual ER remains unchanged; implementation additions are documented here.

GitHub repository

Submission status: ready after genuine member Git contribution and course-specific manual confirmations.

A Domain Tables

CAMPUS

CAMPUS(campus_id PK, campus_name, street, city, state, postal_code)

Column	Type	NULL	Default	Key or generation
campus_id	int unsigned	NO	-	PK; AUTO_INCREMENT
campus_name	varchar(120)	NO	-	-
street	varchar(150)	YES	-	-
city	varchar(80)	YES	-	-
state	varchar(80)	YES	-	-
postal_code	varchar(20)	YES	-	-

Keys, constraints and indexes:

• PRIMARY KEY (campus_id)

• idx_campus_city_name: INDEX (city, campus_name)

DEPARTMENT

DEPARTMENT(department_id PK, department_name, campus_id FK -> CAMPUS.campus_id)

Column	Type	NULL	Default	Key or generation
department_id	int unsigned	NO	-	PK; AUTO_INCREMENT
department_name	varchar(120)	NO	-	-
campus_id	int unsigned	NO	-	FK -> CAMPUS.campus_id

Keys, constraints and indexes:

• PRIMARY KEY (department_id)

• fk_department_campus: (campus_id) -> CAMPUS(campus_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_department_campus: INDEX (campus_id)

• idx_department_name: INDEX (department_name)

USER

USER(user_id PK, first_name, middle_name, last_name, phone, email, role, created_at, department_id FK -> DEPARTMENT.department_id)

Column	Type	NULL	Default	Key or generation
user_id	int unsigned	NO	-	PK; AUTO_INCREMENT
first_name	varchar(80)	NO	-	-
middle_name	varchar(80)	YES	-	-
last_name	varchar(80)	NO	-	-
phone	varchar(20)	YES	-	-
email	varchar(255)	NO	-	-
role	varchar(30)	NO	-	-
created_at	datetime	NO	CURRENT_TIMESTAMP	-
department_id	int unsigned	YES	-	FK -> DEPARTMENT.department_id

Keys, constraints and indexes:

• PRIMARY KEY (user_id)

• fk_user_department: (department_id) -> DEPARTMENT(department_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• uq_user_email: UNIQUE (email)

• chk_user_role: CHECK (upper(trim(role)) in ('STUDENT','STAFF','ADMIN'))

• fk_user_department: INDEX (department_id)

CATEGORY

CATEGORY(category_id PK, category_name, description)

Column	Type	NULL	Default	Key or generation
category_id	int unsigned	NO	-	PK; AUTO_INCREMENT
category_name	varchar(100)	NO	-	-
description	text	YES	-	-

Keys, constraints and indexes:

• PRIMARY KEY (category_id)

• uq_category_name: UNIQUE (category_name)

LOCATION

LOCATION(location_id PK, location_name, building, floor, area, type, campus_id FK -> CAMPUS.campus_id)

Column	Type	NULL	Default	Key or generation
location_id	int unsigned	NO	-	PK; AUTO_INCREMENT
location_name	varchar(150)	NO	-	-
building	varchar(100)	YES	-	-
floor	varchar(50)	YES	-	-
area	varchar(100)	YES	-	-
type	varchar(50)	YES	-	-
campus_id	int unsigned	NO	-	FK -> CAMPUS.campus_id

Keys, constraints and indexes:

• PRIMARY KEY (location_id)

• fk_location_campus: (campus_id) -> CAMPUS(campus_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_location_campus: INDEX (campus_id)

LOST_ITEM

LOST_ITEM(lost_id PK, item_name, description, lost_datetime, color, brand, status, remarks, created_at, reported_by FK -> USER.user_id, category_id FK -> CATEGORY.category_id, location_id FK -> LOCATION.location_id, revision)

Column	Type	NULL	Default	Key or generation
lost_id	int unsigned	NO	-	PK; AUTO_INCREMENT
item_name	varchar(150)	NO	-	-
description	text	YES	-	-
lost_datetime	datetime	NO	-	-
color	varchar(60)	YES	-	-
brand	varchar(100)	YES	-	-
status	varchar(30)	NO	-	-
remarks	text	YES	-	-
created_at	datetime	NO	CURRENT_TIMESTAMP	-
reported_by	int unsigned	NO	-	FK -> USER.user_id
category_id	int unsigned	NO	-	FK -> CATEGORY.category_id
location_id	int unsigned	NO	-	FK -> LOCATION.location_id
revision	int unsigned	NO	1	-

Keys, constraints and indexes:

• PRIMARY KEY (lost_id)

• fk_lost_item_category: (category_id) -> CATEGORY(category_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_lost_item_location: (location_id) -> LOCATION(location_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_lost_item_reporter: (reported_by) -> USER(user_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• chk_lost_item_status: CHECK (upper(trim(status)) in ('OPEN','MATCHED','CLOSED'))

• chk_lost_name: CHECK (char_length(trim(item_name)) >= 2)

• fk_lost_item_location: INDEX (location_id)

• idx_lost_category_location: INDEX (category_id, location_id)

• idx_lost_datetime: INDEX (lost_datetime)

• idx_lost_item_status_datetime: INDEX (status, lost_datetime)

• idx_lost_user_status_created: INDEX (reported_by, status, created_at)

revision starts at 1 and the application increments it when editable core or MongoDB details change. The SQL date triggers reject a timestamp more than one minute in the future.

FOUND_ITEM

FOUND_ITEM(found_id PK, item_name, description, found_datetime, color, brand, status, remarks, created_at, reported_by FK -> USER.user_id, category_id FK -> CATEGORY.category_id, location_id FK -> LOCATION.location_id, revision)

Column	Type	NULL	Default	Key or generation
found_id	int unsigned	NO	-	PK; AUTO_INCREMENT
item_name	varchar(150)	NO	-	-
description	text	YES	-	-
found_datetime	datetime	NO	-	-
color	varchar(60)	YES	-	-
brand	varchar(100)	YES	-	-
status	varchar(30)	NO	-	-
remarks	text	YES	-	-
created_at	datetime	NO	CURRENT_TIMESTAMP	-
reported_by	int unsigned	NO	-	FK -> USER.user_id
category_id	int unsigned	NO	-	FK -> CATEGORY.category_id
location_id	int unsigned	NO	-	FK -> LOCATION.location_id
revision	int unsigned	NO	1	-

Keys, constraints and indexes:

• PRIMARY KEY (found_id)

• fk_found_item_category: (category_id) -> CATEGORY(category_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_found_item_location: (location_id) -> LOCATION(location_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_found_item_reporter: (reported_by) -> USER(user_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• chk_found_item_status: CHECK (upper(trim(status)) in ('OPEN','MATCHED','CLOSED'))

• chk_found_name: CHECK (char_length(trim(item_name)) >= 2)

• fk_found_item_location: INDEX (location_id)

• idx_found_category_location: INDEX (category_id, location_id, status)

• idx_found_item_status_datetime: INDEX (status, found_datetime)

• idx_found_user_created: INDEX (reported_by, created_at)

revision starts at 1 and the application increments it when editable core or MongoDB details change. The SQL date triggers reject a timestamp more than one minute in the future.

MATCH

MATCH(match_id PK, lost_id FK -> LOST_ITEM.lost_id, found_id FK -> FOUND_ITEM.found_id, similarity_score, match_method, status, matched_at, reviewed_by FK -> USER.user_id, reviewed_at, review_remarks, accepted_lost_id GENERATED, accepted_found_id GENERATED, lost_revision, found_revision)

Column	Type	NULL	Default	Key or generation
match_id	int unsigned	NO	-	PK; AUTO_INCREMENT
lost_id	int unsigned	NO	-	FK -> LOST_ITEM.lost_id
found_id	int unsigned	NO	-	FK -> FOUND_ITEM.found_id
similarity_score	decimal(10,6)	NO	-	-
match_method	varchar(100)	NO	-	-
status	varchar(30)	NO	-	-
matched_at	datetime	NO	CURRENT_TIMESTAMP	-
reviewed_by	int unsigned	YES	-	FK -> USER.user_id
reviewed_at	datetime	YES	-	-
review_remarks	text	YES	-	-
accepted_lost_id	int unsigned	YES	-	STORED GENERATED
accepted_found_id	int unsigned	YES	-	STORED GENERATED
lost_revision	int unsigned	NO	1	-
found_revision	int unsigned	NO	1	-

Keys, constraints and indexes:

• PRIMARY KEY (match_id)

• fk_match_found_item: (found_id) -> FOUND_ITEM(found_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_match_lost_item: (lost_id) -> LOST_ITEM(lost_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_match_reviewer: (reviewed_by) -> USER(user_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• uq_match_accepted_found: UNIQUE (accepted_found_id)

• uq_match_accepted_lost: UNIQUE (accepted_lost_id)

• uq_match_pair: UNIQUE (lost_id, found_id)

• chk_match_method: CHECK (upper(trim(match_method)) in ('AI','MANUAL','HYBRID'))

• chk_match_review_relationship: CHECK ((reviewed_by is not null) or ((reviewed_at is null) and (review_remarks is null)))

• chk_match_review_status: CHECK ((upper(trim(status)) = 'PENDING') or (reviewed_by is not null))

• chk_match_similarity_score: CHECK ((similarity_score >= 0) and (similarity_score <= 1))

• chk_match_status: CHECK (upper(trim(status)) in ('PENDING','ACCEPTED','REJECTED'))

• accepted_lost_id: STORED GENERATED AS (case when (status = 'ACCEPTED') then lost_id else NULL end)

• accepted_found_id: STORED GENERATED AS (case when (status = 'ACCEPTED') then found_id else NULL end)

• fk_match_found_item: INDEX (found_id)

• fk_match_reviewer: INDEX (reviewed_by)

• idx_match_lost_status: INDEX (lost_id, status)

• idx_match_similarity: INDEX (similarity_score)

• idx_match_status: INDEX (status)

lost_revision and found_revision snapshot report versions. sp_review_match checks them before acceptance. For canonical ACCEPTED rows, generated unique report IDs allow at most one accepted match per lost report and per found report. Pending/rejected rows have NULL generated values. Review-role triggers require a STAFF or ADMIN reviewer.

CLAIM

CLAIM(claim_id PK, claim_description, status, created_at, claim_remarks, submitted_by FK -> USER.user_id, found_id FK -> FOUND_ITEM.found_id, processed_by FK -> USER.user_id, processed_at, processing_remarks, approved_found_id GENERATED)

Column	Type	NULL	Default	Key or generation
claim_id	int unsigned	NO	-	PK; AUTO_INCREMENT
claim_description	text	NO	-	-
status	varchar(30)	NO	-	-
created_at	datetime	NO	CURRENT_TIMESTAMP	-
claim_remarks	text	YES	-	-
submitted_by	int unsigned	NO	-	FK -> USER.user_id
found_id	int unsigned	NO	-	FK -> FOUND_ITEM.found_id
processed_by	int unsigned	YES	-	FK -> USER.user_id
processed_at	datetime	YES	-	-
processing_remarks	text	YES	-	-
approved_found_id	int unsigned	YES	-	STORED GENERATED

Keys, constraints and indexes:

• PRIMARY KEY (claim_id)

• fk_claim_found_item: (found_id) -> FOUND_ITEM(found_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_claim_processor: (processed_by) -> USER(user_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• fk_claim_submitter: (submitted_by) -> USER(user_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• uq_claim_approved_found: UNIQUE (approved_found_id)

• uq_claim_user_found: UNIQUE (submitted_by, found_id)

• chk_claim_description: CHECK (char_length(trim(claim_description)) >= 10)

• chk_claim_process_relationship: CHECK ((processed_by is not null) or ((processed_at is null) and (processing_remarks is null)))

• chk_claim_processing_status: CHECK ((upper(trim(status)) = 'PENDING') or (processed_by is not null))

• chk_claim_status: CHECK (upper(trim(status)) in ('PENDING','APPROVED','REJECTED'))

• approved_found_id: STORED GENERATED AS (case when (status = 'APPROVED') then found_id else NULL end)

• fk_claim_found_item: INDEX (found_id)

• fk_claim_processor: INDEX (processed_by)

• idx_claim_status: INDEX (status)

• idx_claim_user_created: INDEX (submitted_by, created_at)

For canonical APPROVED rows, approved_found_id permits at most one approved claim per found report. Other rows have NULL. Processor-role triggers require STAFF or ADMIN. The application/procedure also requires the accepted lost owner and prohibits processing one's own claim.

NOTIFICATION

NOTIFICATION(notification_id PK, title, message, type, is_read, created_at, user_id FK -> USER.user_id)

Column	Type	NULL	Default	Key or generation
notification_id	int unsigned	NO	-	PK; AUTO_INCREMENT
title	varchar(200)	NO	-	-
message	text	NO	-	-
type	varchar(50)	NO	-	-
is_read	tinyint(1)	NO	0	-
created_at	datetime	NO	CURRENT_TIMESTAMP	-
user_id	int unsigned	NO	-	FK -> USER.user_id

Keys, constraints and indexes:

• PRIMARY KEY (notification_id)

• fk_notification_user: (user_id) -> USER(user_id); ON UPDATE RESTRICT; ON DELETE RESTRICT

• chk_notification_is_read: CHECK (is_read in (0,1))

• chk_notification_type: CHECK (upper(trim(type)) in ('MATCH','CLAIM','GENERAL'))

• idx_notification_user_read: INDEX (user_id, is_read)

• idx_notification_user_read_created: INDEX (user_id, is_read, created_at)

B Infrastructure and Internal Tables

MONGO_OUTBOX

MONGO_OUTBOX(outbox_id PK, collection_name, document_id, payload, operation, created_at, processed_at, attempts, last_error)

Column	Type	NULL	Default	Key or generation
outbox_id	bigint unsigned	NO	-	PK; AUTO_INCREMENT
collection_name	varchar(40)	NO	-	-
document_id	int unsigned	NO	-	-
payload	json	NO	-	-
operation	varchar(10)	NO	UPSERT	-
created_at	datetime	NO	CURRENT_TIMESTAMP	-
processed_at	datetime	YES	-	-
attempts	int unsigned	NO	0	-
last_error	varchar(500)	YES	-	-

Keys, constraints and indexes:

• PRIMARY KEY (outbox_id)

• chk_outbox_collection: CHECK (collection_name in ('lost_item_details','found_item_details','match_evidence'))

• chk_outbox_operation: CHECK (operation in ('UPSERT','DELETE'))

• idx_outbox_pending: INDEX (processed_at, outbox_id)

This internal table stores ordered MongoDB UPSERT/DELETE payloads. document_id is not a SQL foreign key because its target depends on collection_name and a DELETE can outlive its domain row. Application services validate linked IDs.

SCHEMA_MIGRATION

SCHEMA_MIGRATION(name PK, checksum, applied_at)

Column	Type	NULL	Default	Key or generation
name	varchar(100)	NO	-	PK
checksum	char(64)	NO	-	-
applied_at	datetime	NO	CURRENT_TIMESTAMP	-

Keys, constraints and indexes:

• PRIMARY KEY (name)

This internal table tracks applied migration names, checksums and timestamps. The current setup checks whether a migration name already exists; it does not compare an existing checksum or automatically reverse a partly applied DDL migration.

Relationship cardinalities

• CAMPUS 1 to many DEPARTMENT and LOCATION; each child campus_id is required.

• DEPARTMENT 1 to many USER; USER.department_id is optional.

• USER 1 to many LOST_ITEM, FOUND_ITEM, CLAIM submissions and NOTIFICATION recipients.

• CATEGORY and LOCATION each classify or locate many lost/found reports; report references are required.

• Each LOST_ITEM and FOUND_ITEM can have many candidate MATCH rows, but at most one canonical ACCEPTED match per report.

• USER 1 to many MATCH reviews and CLAIM processing actions; actor fields are nullable before a decision and role-checked when supplied.

• FOUND_ITEM 1 to many CLAIM rows; one per submitter/found pair and at most one canonical APPROVED claim per found item.

• MONGO_OUTBOX and SCHEMA_MIGRATION have no SQL foreign keys. MongoDB links use the SQL IDs and are checked by the application, not cross-database FK constraints.

Verified inventory: 12 SQL base tables (10 domain and 2 internal), 2 views, 2 stored procedures, 1 stored function, 8 triggers and 4 MongoDB collections. Revisions are application-maintained fields; the database does not auto-increment them with a trigger. Terminal state rules are enforced through the APIs and stored procedures; raw SQL access must be restricted.

Full installed metadata: final-schema.json (../testing/results/final-schema.json). Executable sources: schema.sql, indexes_views.sql, advanced.sql, revisions.sql, procedures.sql and setup-databases.mjs.